COMP2007: Operating Systems & Concurrency
Week 7 – 12:00pm Wednesday – 06 November 2024

valid for 75 minutes from 11:45am
generated 2024-09-20 14:34

Figure: Attendance Monitoring

Operating Systems and Concurrency

Memory Management 3

COMP2007

Geert De Maere
(Dan Marsden)
{Geert.DeMaere,Dan.Marsden}@Nottingham.ac.uk

University Of Nottingham
United Kingdom

2024

Overview

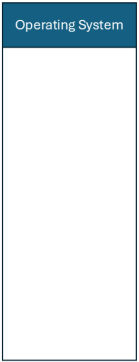
Goals for Today

- **Bitmaps** and **linked lists** for free space management
- **Allocation Policies:**
  - First fit, best fit, next fit, worst fit, quick fit
  - Buddy algorithm
- **Preventing fragmentation** - **non-contiguous** memory management
  - **Paging** - page tables, address translation
  - **Segmentation**

Recall

Last Lecture

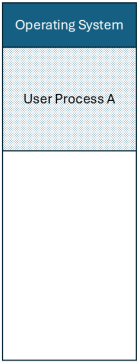
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

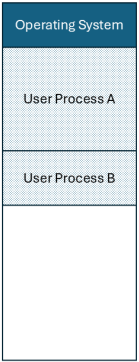
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

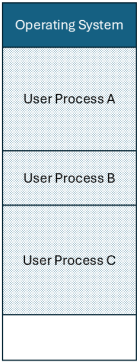
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

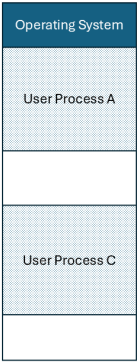
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

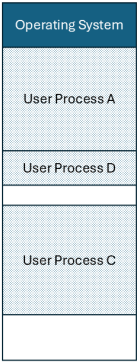
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

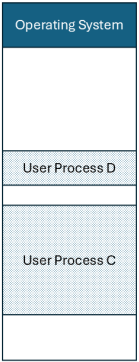
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

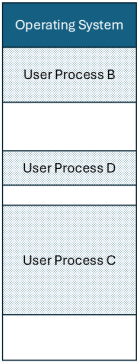
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

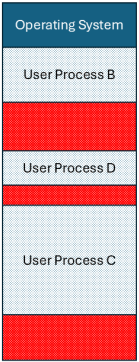
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

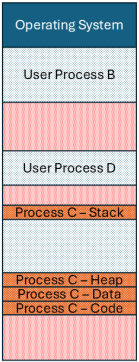
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

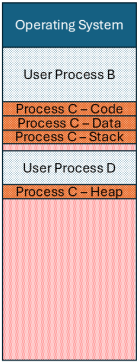
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

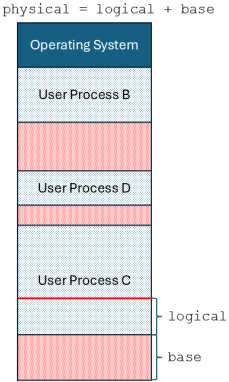
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

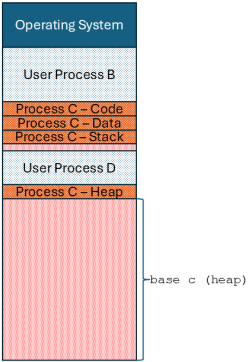
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

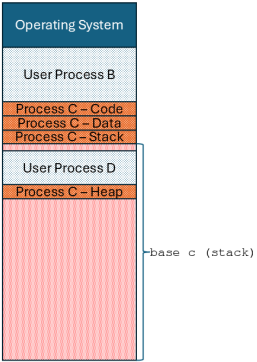
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

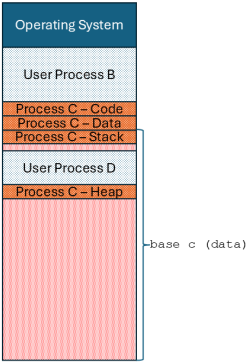
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

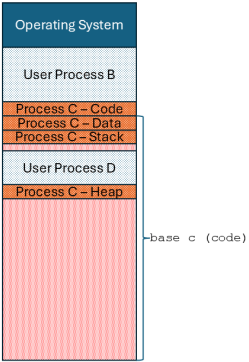
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

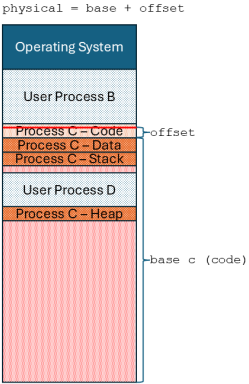
- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

- **Dynamic partitioning**, three key problems:
  - External Fragmentation
  - Free space management
  - Sparse logical address space
- **Segmentation**:
  - Offers “some” relief from external fragmentation
  - Requires a base and bound register per segment



Recall

Last Lecture

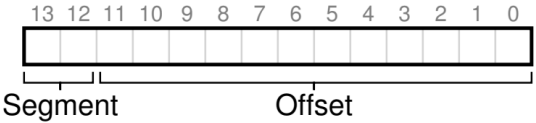


Figure: Logical address for segmentation (offset = position relative to start of segment)

Segment	index	Base	Bound	RWX
Code	00	...	...	...
Data	01	...	...	...
Heap	10	...	...	...
Stack	11	...	...	...

Dynamic Partitioning

What is Required?

- **Reducing fragmentation** and handling **sparse address** spaces (segmentation)
- A method to **keep track** of free memory
- A strategy/policy to **allocate free** memory when requested by a process

Managing Free Space

Allocation Structures: Bitmaps

- The simplest data structure that can be used is a form of **bitmap**
- **Memory** is **split into blocks** of say 4KB size (2^{12} bytes)
  - A bit map is set up so that each **bit is 0** if the **memory block is free** and **1** is the **block is used**, e.g.
    - 32MB memory $\Rightarrow$ $32*2^{20}$ / $2^{12} = 32 \times 2^8$ bitmap entries (8192)
    - 8192 bits occupy 8192 / 8 = 1KB of storage (only!)
  - The size of this bitmap will depend on the **size of the memory** and **the size of the allocation unit**.

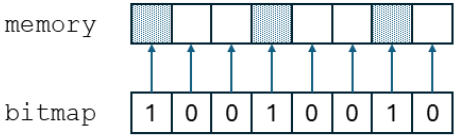


Figure: Memory management with bitmaps

Managing Free Space

Allocation Structures: Bitmaps (Cont'ed)

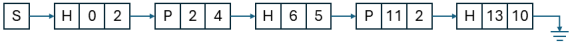
- To find a hole of e.g. size 128KB, then a group of **32 adjacent bits set to zero** must be found, typically a **long operation** (esp. with smaller blocks)
- A **trade-off exists** between the **size of the bitmap** and the **size of blocks** exists
  - The **size of bitmaps** can become prohibitive for small blocks and may **make searching** the bitmap **slower**
  - Larger blocks may increase **internal fragmentation**
- **Bitmaps** are **rarely used** for these reasons

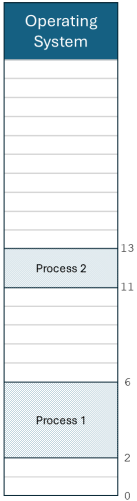
©University of Nottingham

Managing Free Space

Allocation Structures: Linked List

- A linked list enables to deal with **a variable number** of **free and used partitions**
  - A linked list consists of a **number of entries** (“links”!)
  - Each link **contains data items**, e.g. **start of memory block**, **size**, free/allocated **flag**
  - Each link also contains a **pointer to the next** in the chain
- The **allocation** of processes to unused blocks becomes **non-trivial**





Managing Free Space

Allocation Structures

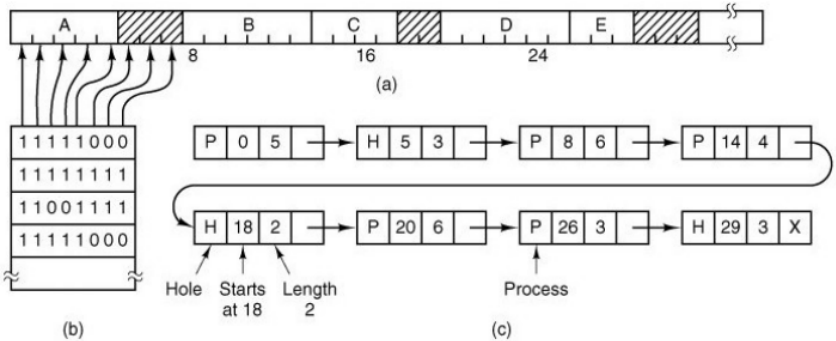
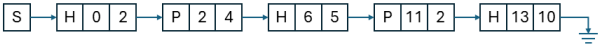


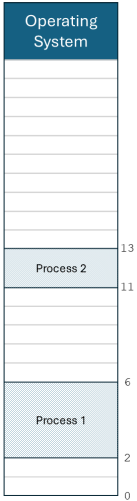
Figure: (a) A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap. (c) The same information as a list. (from Tanenbaum)

Memory Allocation

First Fit

- **First fit** scans **from the start** of the list until an **sufficiently large gap** is found
  - `if` the space is **the exact size** then all the space is **allocated**
  - `else` the space is **split**:
    - The first entry is set to the **size requested** and marked "**used**"
    - The second entry is set to **remaining size** and marked "**free**"



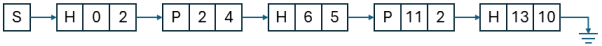


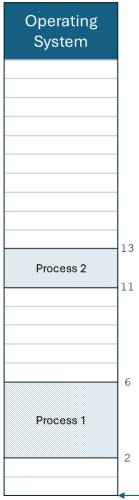
©University of Nottingham

Memory Allocation

First Fit

- **First fit** scans **from the start** of the list until an **sufficiently large gap** is found
  - `if` the space is **the exact size** then all the space is **allocated**
  - `else` the space is **split**:
    - The first entry is set to the **size requested** and marked "**used**"
    - The second entry is set to **remaining size** and marked "**free**"



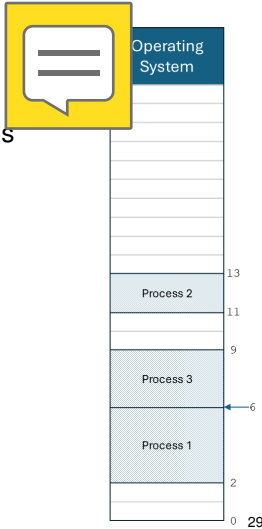


©University of Nottingham

Memory Allocation

First Fit

- **First fit** scans **from the start** of the list until an **sufficiently large gap** is found
  - `if` the space is **the exact size** then all the space is **allocated**
  - `else` the space is **split**:
    - The first entry is set to the **size requested** and marked "**used**"
    - The second entry is set to **remaining size** and marked "**free**"

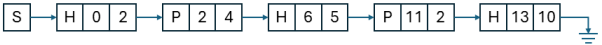


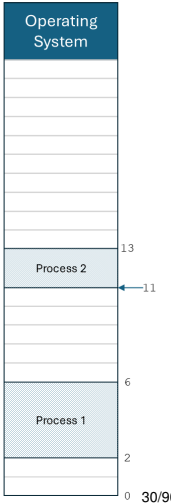
©University of Nottingham

Memory Allocation

Next Fit

- The **next fit algorithm** maintains a record of where it got to:
  - It **restarts its search** from **where it stopped last time**
  - It gives an **even chance to all memory** to get allocated (first fit concentrates on the start of the list)
- Simulations have shown that next fit **performs worse** than first fit



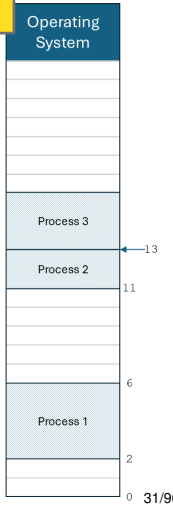


©University of Nottingham

Memory Allocation

Next Fit

- The **next fit algorithm** maintains a record of where it got to:
  - It **restarts its search** from **where it stopped last time**
  - It gives an **even chance to all memory** to get allocated (first fit concentrates on the start of the list)
- Simulations have shown that next fit **performs worse** than first fit



©University of Nottingham

Memory Allocation

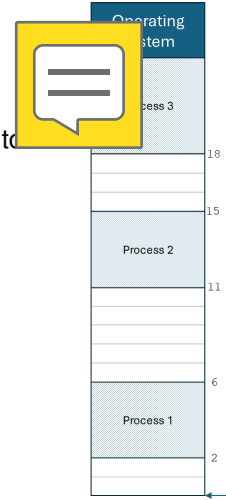
First vs. Next Fit

- **First fit** is a fast and looks for the **first available hole**
  - It does **not account** for a **better fitting** hole later
  - It may **break up** a **large holes** early in the list
- **Next fit** does not change this

Memory Allocation

Best Fit

- **Best fit** searches the **entire linked** list for the **smallest hole** big enough to satisfy the request
  - It is **slower** than first fit
  - Result in **small leftover holes** (wastes memory)

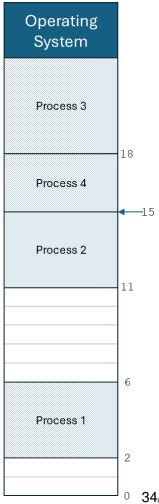


©University of Nottingham

Memory Allocation

Best Fit

- **Best fit** searches the **entire linked** list for the **smallest hole** big enough to satisfy the request
  - It is **slower** than first fit
  - Result in **small leftover holes** (wastes memory)

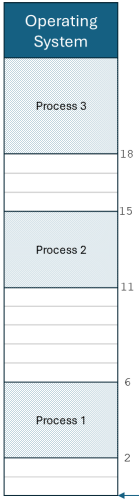


©University of Nottingham

Memory Allocation

Worst Fit

- **Best fit** split an empty partition in **small holes**
- **Worst fit** finds the **largest available partition** and splits it
  - The left over part **will still be large** (potentially more useful)
  - Simulations have shown that **worst fit** is **not good** in practice

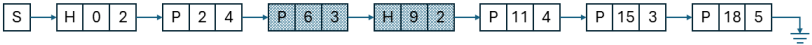


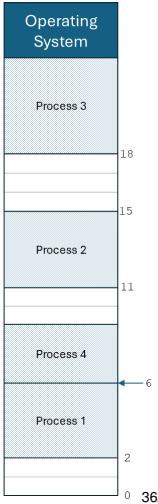
©University of Nottingham

Memory Allocation

Worst Fit

- **Best fit** split an empty partition in **small holes**
- **Worst fit** finds the **largest available partition** and splits it
  - The left over part **will still be large** (potentially more useful)
  - Simulations have shown that **worst fit** is **not good** in practice





©University of Nottingham

Memory Allocation

Summary

- **First fit**: allocate **first block** that is **large enough**
- **Next fit**: allocate **next block** that is large enough, i.e. **starting from the current location**
- **Best fit**: choose block that **matches** required size **closest** - O(N) complexity
- **Worst fit**: choose the **largest possible block** - O(N) complexity

©University of Nottingham

Dynamic Partitioning

Allocating Available Memory: Quick Fit and Others

- **Quick fit** maintains **lists of commonly used sizes**
  - For example a separate list for each of 4K, 8K, 12K, 16K, etc., holes
  - **Odd sizes** can either go into the **nearest size** or into a **special separate list**
- It is **much faster** to find the required size hole using **quick fit**
- Similar to **best fit**, it has the problem of creating **many tiny holes**

Dynamic Partitioning

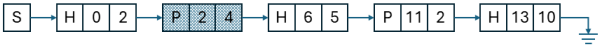
Allocating Available Memory: Quick Fit and Others

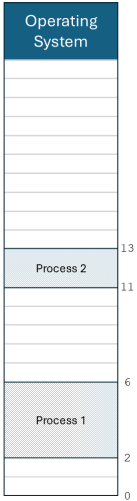
- **Quick fit** maintains **lists of commonly used sizes**
  - For example a separate list for each of 4K, 8K, 12K, 16K, etc., holes
  - **Odd sizes** can either go into the **nearest size** or into a **special separate list**
- It is **much faster** to find the required size hole using **quick fit**
- Similar to **best fit**, it has the problem of creating **many tiny holes**
- Finding neighbours for **coalescing** (combining empty partitions) becomes more difficult/time consuming

Memory Allocation

Coalescing

- **Coalescing** (joining together) takes place when two **adjacent entries** in the linked list **become free**
- Both **neighbours** are examined when a **block is freed**
  - `if` either (or both) are **also free**
  - `else` the two (or three) **entries are combined** into one larger block by adding up the sizes
    - The earlier block in the linked list gives the **start point**
    - The **separate links are deleted** and a single link inserted



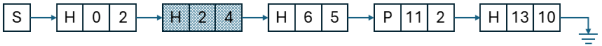


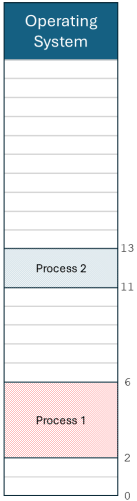
©University of Nottingham

Memory Allocation

Coalescing

- **Coalescing** (joining together) takes place when two **adjacent entries** in the linked list **become free**
- Both **neighbours** are examined when a **block is freed**
  - `if` either (or both) are **also free**
  - `else` the two (or three) **entries are combined** into one larger block by adding up the sizes
    - The earlier block in the linked list gives the **start point**
    - The **separate links are deleted** and a single link inserted



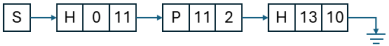


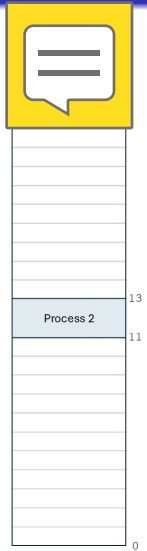
©University of Nottingham

Memory Allocation

Coalescing

- **Coalescing** (joining together) takes place when two **adjacent entries** in the linked list **become free**
- Both **neighbours** are examined when a **block is freed**
  - `if` either (or both) are **also free**
  - `else` the two (or three) **entries are combined** into one larger block by adding up the sizes
    - The earlier block in the linked list gives the **start point**
    - The **separate links are deleted** and a single link inserted





©University of Nottingham

Memory Allocation

Compacting

- Even with coalescing, **free blocks** may still **distributed** across memory
- **Compacting** can be used but is harder to implement and **time consuming**
- Processes may have to be **moved/swapped out**, **free space coalesced**, and processes **swapped back in** at lowest available location

©University of Nottingham

(Semi-)Contiguous Allocation Schemes

Overview and Shortcomings

- Different memory management schemes have different advantages/disadvantages
  - **Mono-programming** is easy but does result in **low resource utilisation**
  - **Fixed partitioning** facilitates **multi-programming** but results in **internal fragmentation**
  - **Dynamic partitioning & segmentation** facilitate **multi-programming**, reduce **internal fragmentation**, but result in **external fragmentation** (allocation methods, coalescing, and compacting alleviate)
- Can we design a memory management scheme that **resolves the shortcomings** of **(semi-)contiguous memory schemes**?

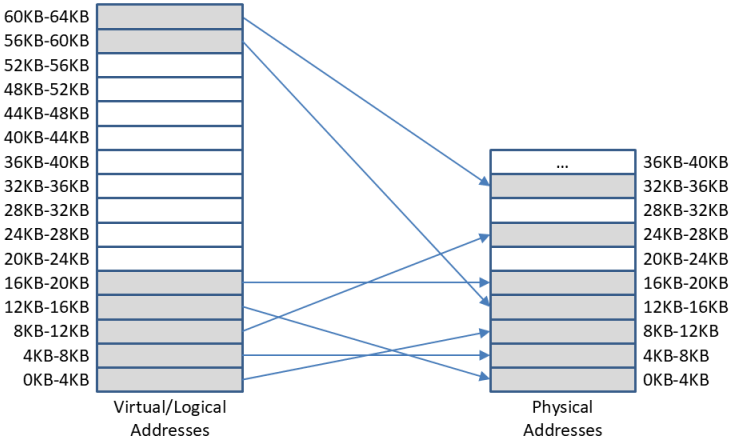
Paging

Principles

- **Paging** uses **fixed partitioning** and **code re-location** to devise a new ***non-co*** management scheme:
  - Memory is split into **much smaller blocks**
  - A process is allocated **one or more blocks** (e.g., a 11KB process would take up three 4KB blocks)
  - Blocks do **not** have to be stored **in contiguous in physical memory**
  - The process **perceives** them to be **contiguous**
- Benefits of non-contiguous schemes include:
  - **Internal fragmentation** is reduced to **a few last "blocks"** only
  - **No external fragmentation** (blocks in physical address space are **stacked** directly on top of each other)

Paging

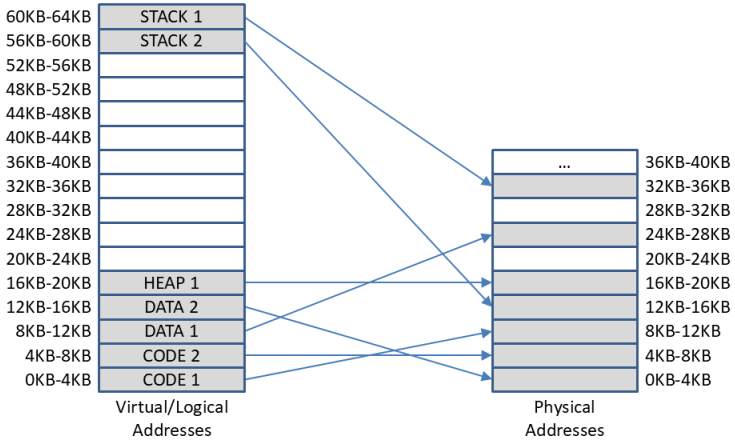
Principles (Cont'ed)



©University of Nottingham

Paging

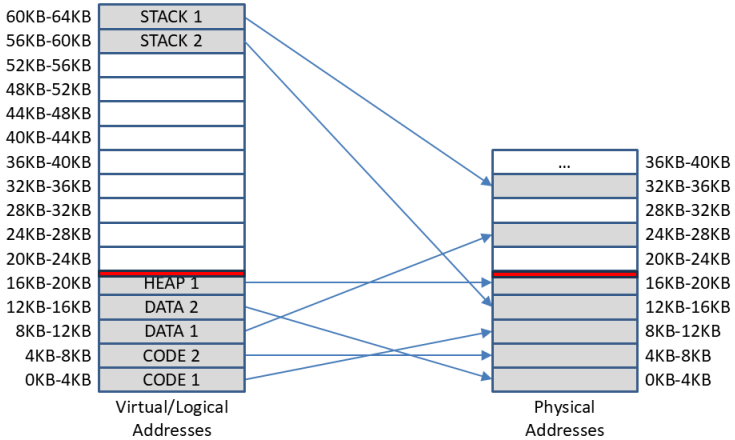
Principles (Cont'ed)



©University of Nottingham

Paging

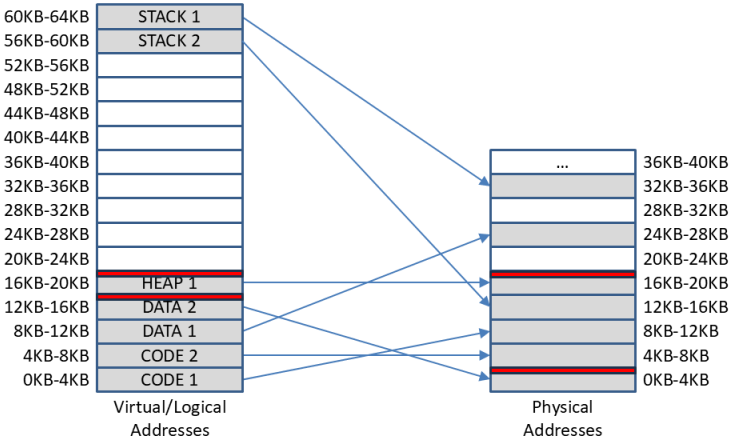
Principles (Cont'ed)



©University of Nottingham

Paging

Principles (Cont'ed)



©University of Nottingham

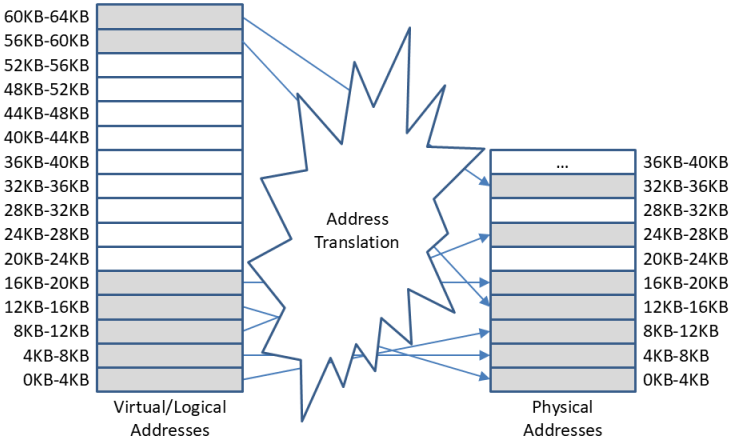
Paging

Principles: Definitions

- A **page** is a small block of **contiguous memory** in the **logical address space**, i.e. as seen by the process
- A **frame** is a **small contiguous block** in **physical memory**
- Pages and frames (commonly) have the **same size**:
  - The size is usually a power of 2
  - Sizes range between 512 bytes and 1Gb
- Pages are **mapped** on to frames

Paging

Relocation



©University of Nottingham

Segmentation

Segmentation Table

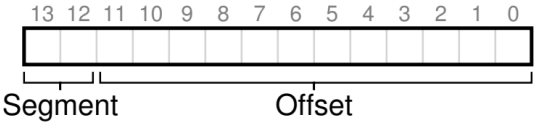


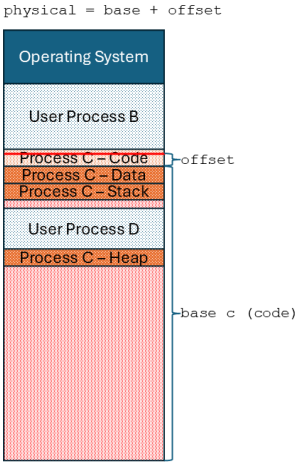
Figure: Logical address for segmentation (offset = position relative to start of segment)

Segment	index	Base	Bound	RWX
Code	00	...	...	...
Data	01	...	...	...
Heap	10	...	...	...
Stack	11	...	...	...

©University of Nottingham

Segmentation

Segmentation Table



Paging

Page Table

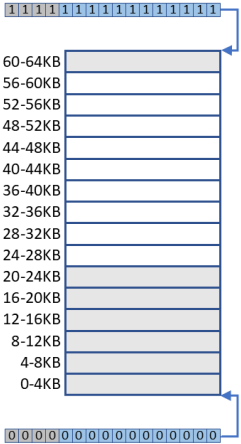
Figure: Logical address for page (offset = position relative to start of the page)

Page Number	Frame Number	...	...	RWX
0000	1010	...	...	...
0001	0100	...	...	...
0010	1111	...	...	...
...	...	...	...	...

Recall

Memory as a Linear Array

- Memory is a **linear array** of **bytes**
- N **address lines** are used to specify 2^N addresses, e.g., 2^{16} for a 16 bit machine
- If each **memory cell** is a **byte**, we can address up to 64KB
- If the memory is split into 16 pages (=2^4), then block size = $2^{16}/2^4 = 2^{12}$ = 4KB



Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

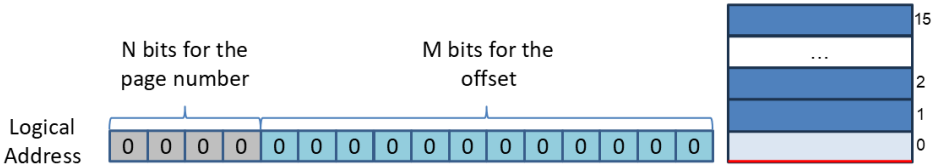


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

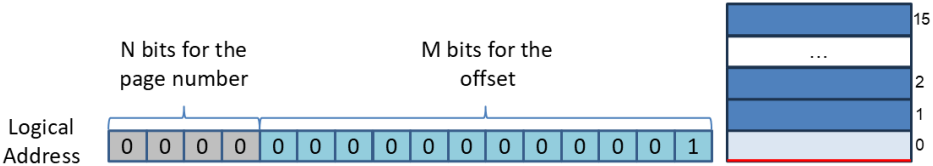


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

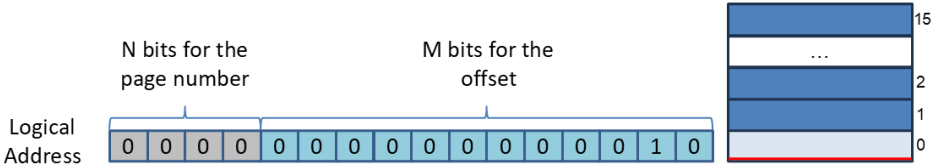


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

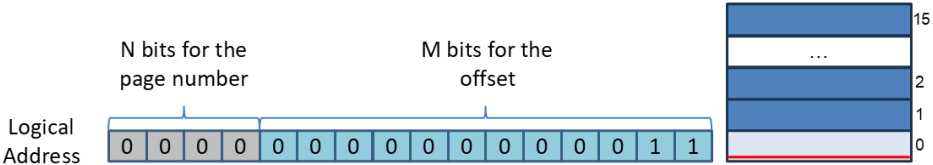


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

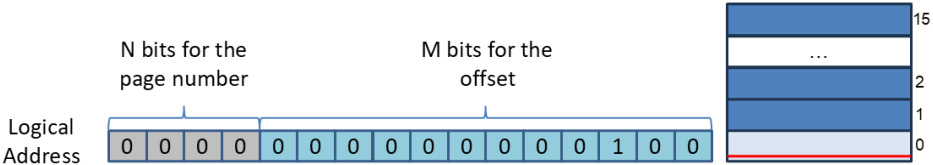


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

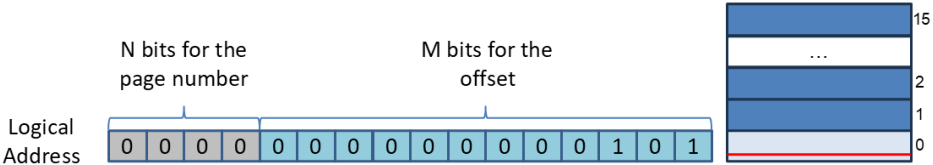


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

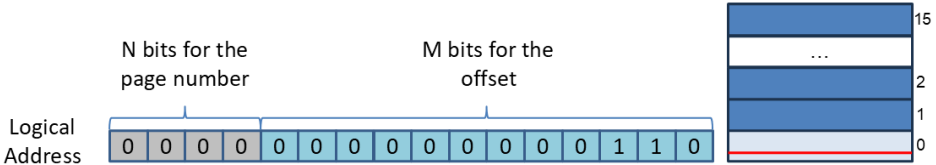


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

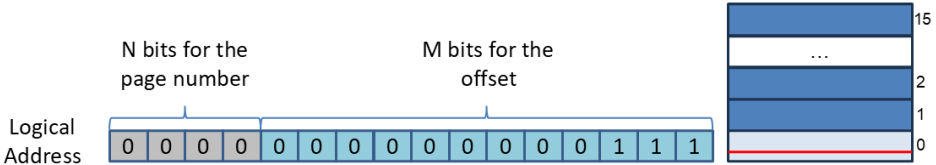


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

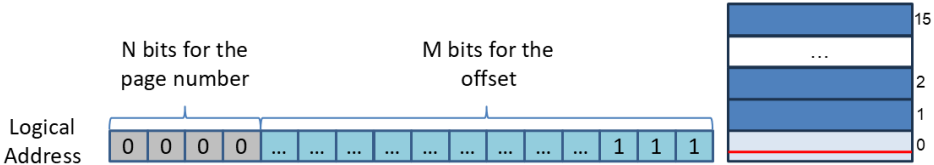


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

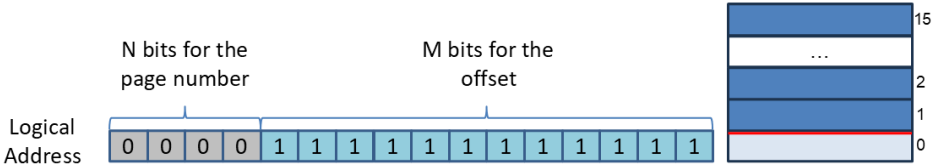


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

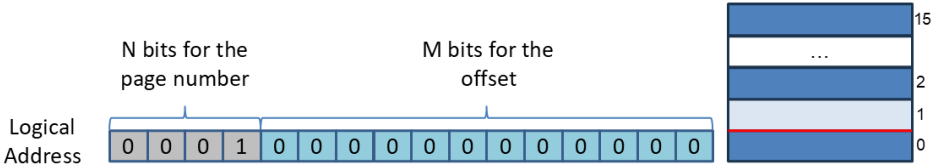


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

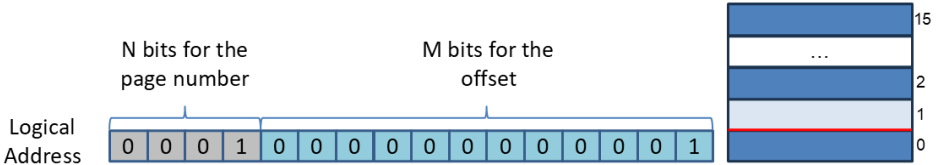


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

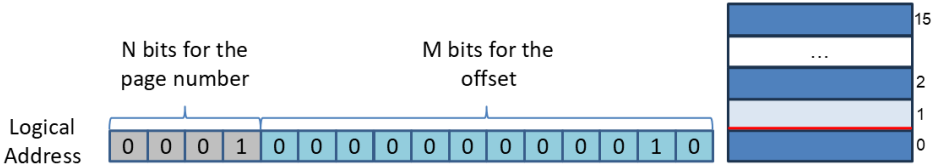


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

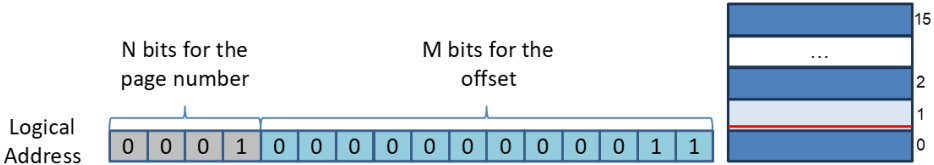


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

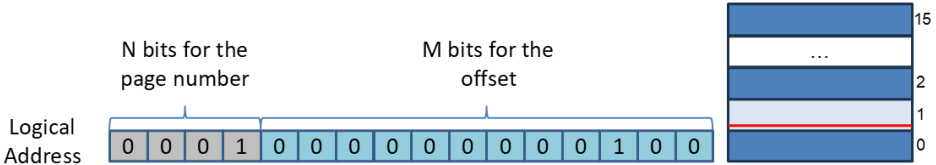


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

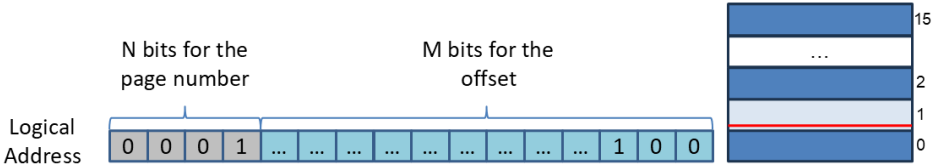


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

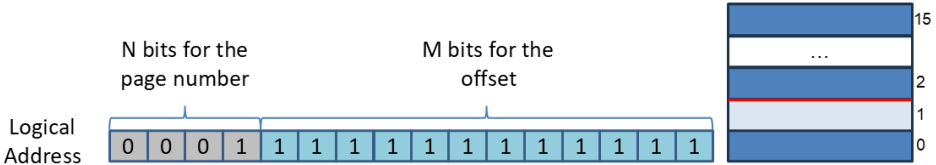


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

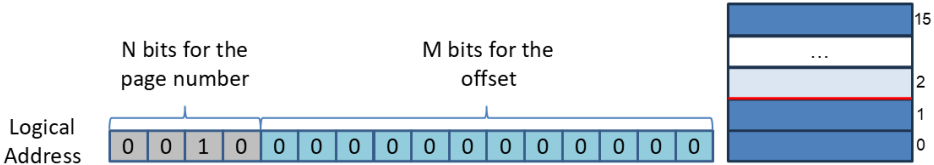


Figure: Paging

Paging

Address Translation: Implementation

- A **logical address** is relative to the start of the **program**
- The **left most** *n* **bits** represent the **page number**
  - e.g. 4 bits for the page number ⇒ 16 pages
- The **right most** *m* **bits** represent the **offset within the page**
  - e.g. 12 bits for the offset ⇒ 4KB pages

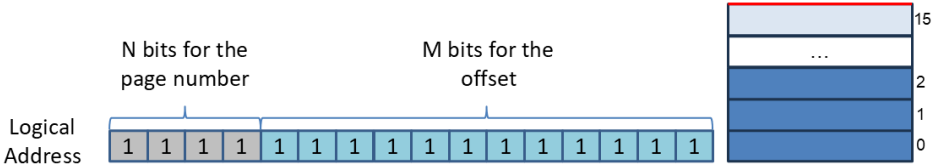


Figure: Paging

Paging

Page Table

Figure: Logical address for page (offset = position relative to start of the page)

Page Number	Frame Number	...	...	RWX
0000	1010	...	...	...
0001	0100	...	...	...
0010	1111	...	...	...
...	...	...	...	...

Paging

Page Table

Figure: Logical address for page (offset = position relative to start of the page)

Page Number	Frame Number	...	...	RWX
0000	1010	...	...	...
0001	0100	...	...	...
0010	1111	...	...	...
...	...	...	...	...

Paging

Page Table

Figure: Logical address for page (offset = position relative to start of the page)

Page Number	Frame Number	...	...	RWX
0000	1010	...	...	...
0001	0100	...	...	...
0010	1111	...	...	...
...	...	...	...	...

Paging

Page Table

Figure: Logical address for page (offset = position relative to start of the page)

Page Number	Frame Number	...	...	RWX
0000	1010	...	...	...
0001	0100	...	...	...
0010	1111	...	...	...
...	...	...	...	...

Paging

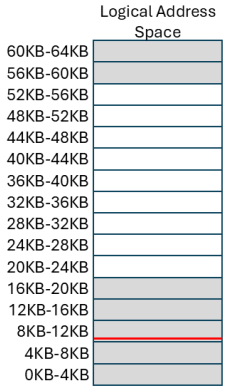
Page Table

Figure: Logical address for page (offset = position relative to start of the page)

Page Number	Frame Number	...	...	RWX
0000	1010	...	...	...
0001	0100	...	...	...
0010	1111	...	...	...
...	...	...	...	...

Paging

Address Translation: Implementation



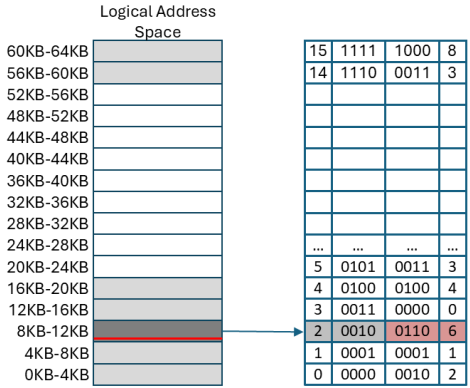
Logical Address

0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0

©University of Nottingham

Paging

Address Translation: Implementation



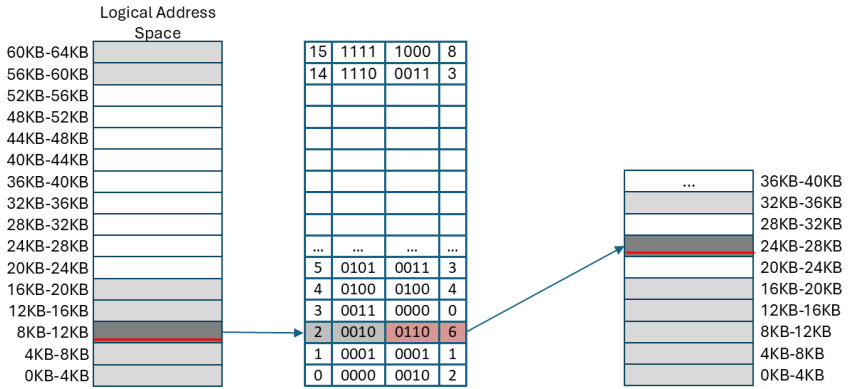
Logical Address

0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0

©University of Nottingham

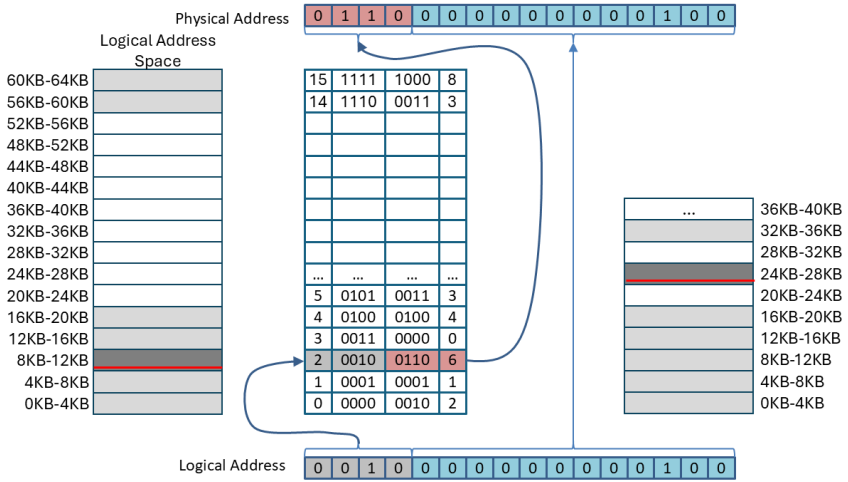
Paging

Address Translation: Implementation



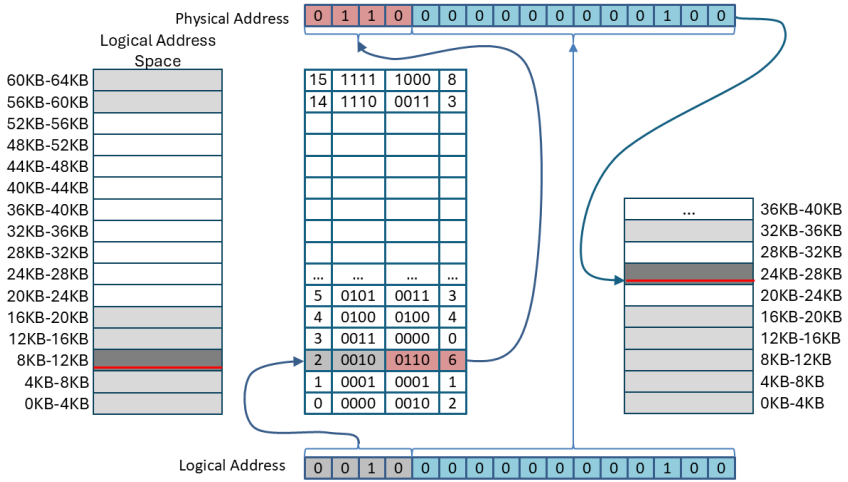
Paging

Address Translation: Implementation



Paging

Address Translation: Implementation



Paging

Address Translation: Implementation

- A **logical (physical) address** consists of two parts:
  - The **right most** *m* **bits** that represent the **offset within the page (frame)**
    - e.g. 12 bits for the offset, allowing up to 4096 (2^{12}) bytes per page (frame)
  - The **left most** *n* **bits** that represent the **page (frame) number**
    - e.g. 4 bits for the page number allowing 16 (2^4) pages (frames)

Paging

Address Translation: Implementation

- **Logical address** needs to be **translated** into a **physical address**
  - The **page table** holds the page to frame mapping
    - The **page number** is used as **index to the page table** that lists the **number of the associated frame**
    - `frameNumber=f(pageNumber)`
  - The **relative location** (offset) within the page and frame **remains the same**
  - The **page number** is replaced with the **frame number**

Paging

Relocation

- **Multiple "base registers"** will be required for a process, one for every page
- The **set of base registers** is stored in the **page table**
- Every process has its **own page table** containing its own **set of "base registers"**
- The **operating system** maintains a **list of free frames**

Recap

Take-Home Message

- Memory **allocation**, **coalescing** and **compacting** in dynamic partitioning
- **Paging**, **page tables**, and **address translation**

Test your Understanding

Tracking Free Memory

Exercise

Compare the **memory needed** to keep track of **free memory** using **bitmap** vs. **linked list**. The size of main memory is **8GB** and the block size is **1MB**. You can assume that exactly **half of the memory is in use**, and that it contains an **alternating sequence** of occupied and free blocks. The linked list only keeps track of **free blocks**, and each node requires a 32-bit memory address, a 16-bit length, and a 32-bit next-node field.

How many bytes of storage are required for each method?

Test Your Understanding

Page tables and memory addressing

Page Tables

Given a **64-bit machine** that uses paging, and a page/frame size of **4KB**.

- What would be the maximum **number of frames**?
- How many **entries** will we have in the **page table**?
- How many **pages** do we have in a **17KB process**?
- How much **memory is wasted** in the last frame?